SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME:Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I R.Yaswanth (192312408) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Yaswanth

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Question 1: AI Drones for Emergency Medicine Delivery (Greedy Best-First Search and A*)

Scenario understanding

The drone must reach a flood-affected destination quickly. It knows a heuristic h(n) (straight-line distance to the goal), receives only partial real-time updates about blocked paths, and faces variable edge costs g(n) because terrain and weather change. The environment is dynamic, so the plan can become invalid mid-flight.

i. Greedy Best-First Search (GBFS)

GBFS expands the node with the lowest heuristic value only: f(n) = h(n). The drone would always move to the neighbouring waypoint that looks closest to the destination in a straight line.

Strengths	Weaknesses under uncertainty
Very fast; expands few nodes, so decisions are almost instant (good for urgency).	Ignores the cost already spent g(n), so it may choose a short-looking path through storms or bad terrain.
Low memory and simple to implement.	Not optimal and not guaranteed complete in graphs with loops or dead ends; can get trapped behind a blocked road or flooded valley.
Works acceptably when the heuristic is accurate and few obstacles exist.	Straight-line distance becomes misleading when roads are blocked, so GBFS is easily fooled.

ii. A* Search

A* uses f(n) = g(n) + h(n), where g(n) is the actual cost travelled so far (including terrain and weather penalties) and h(n) is the estimated remaining cost. Balancing both means it avoids paths that are short in distance but expensive in cost, and it also avoids wandering without progress toward the goal. If h(n) is admissible (never overestimates), A* is complete and returns the least-cost route. When a path is blocked, the node is removed or its cost set to infinity and A* simply continues from the open list or replans, so it can pivot to the next best option. Variable costs are directly encoded in g(n), which is a major advantage in this scenario.

iii. Impact of heuristic accuracy

Aspect	Greedy Best-First	A*
Accurate heuristic	Fast and near-direct path, but still no optimality guarantee.	Expands very few nodes and finds the optimal path.
Underestimating (admissible but loose)	Still goes toward the goal, may be slow.	Optimal but explores more nodes (slower).

Overestimating (inadmissible)	Behaviour largely unaffected, still greedy.	Loses optimality; may return a risky or costly path.
Misleading (blocked roads, detours)	Highly sensitive; may head into dead ends and backtrack heavily.	More robust, because g(n) corrects the error.

Straight-line distance is admissible (no real route is shorter than a straight line) but becomes weak when floods force long detours. A better heuristic could add weather or terrain penalty factors while staying admissible.

iv. Effect of dynamic changes (blocked paths)

- Previously computed paths can become invalid, so the drone must **replan** repeatedly.
- GBFS replans cheaply but may repeat poor choices because it has no memory of cost; it can oscillate between candidate nodes.
- A* replanning from scratch is costlier in time, but variants such as **D* Lite / Lifelong Planning A* / Anytime Repairing A*** reuse earlier search information and update only affected nodes.
- Partial updates mean the drone should treat unknown edges optimistically or with a risk penalty, and re-evaluate when new sensor data arrives.
- Frequent changes increase search time and decision latency; local replanning (re-searching only a nearby window) keeps latency low.

v. Recommendation

Recommended: A* (preferably an incremental or replanning variant such as D* Lite), using a weighted or risk-aware heuristic.

- **Real-time constraint:** GBFS is faster per decision, but A* with a good heuristic is fast enough and can be run in bounded time (weighted A* or anytime A* trade a little optimality for speed).
- **Optimality:** A* minimizes total cost including terrain and weather, saving fuel and delivery time.
- **Safety:** A* accounts for risk by adding hazard costs to g(n); GBFS ignores them and may fly into dangerous zones.
- Use GBFS only as a quick fallback when a decision is needed within milliseconds.

Question 2: Smart City Traffic Signal Optimization (Hill Climbing, Simulated Annealing, Genetic Algorithm)

Problem formulation as an optimization problem

- **State:** a complete assignment of signal timings (cycle length, green split, offset) for every intersection.
- **Decision variables:** green/red durations and phase offsets for hundreds of intersections.
- **Objective function:** minimize C = w1 (total waiting time) + w2 (fuel consumption) + w3 (congestion index), with weights w1, w2, w3 reflecting policy priorities.
- **Constraints:** minimum and maximum green times, pedestrian crossing time, emergency vehicle priority.
- **Neighbourhood (successor function):** change one intersection's timing slightly, or shift an offset.

Why traditional search is inefficient: with n intersections each having k possible timings, the space has k^n states (combinatorial explosion). Systematic search (BFS, DFS, A*) cannot enumerate it, there is no clear goal state (only better or worse solutions), the path to a solution is irrelevant, and traffic changes constantly so stored solutions go stale. Local search, which keeps only the current state and improves it, uses constant memory and suits this problem.

i. Hill Climbing and its limitations

Hill climbing starts with a (random or current) timing plan, evaluates all neighbouring plans, moves to the best neighbour that lowers the cost, and stops when no neighbour is better. It is simple, memory-efficient, and fast to react to small traffic changes. Its limitations: it is greedy, may stop at a local optimum, cannot escape plateaus, depends heavily on the starting plan, and gives no guarantee of a global optimum.

ii. Local maxima, plateaus and ridges (real-world meaning)

Issue	Meaning	Traffic example
Local maximum (optimum)	A state better than all neighbours but worse than the best overall.	Timings at one junction are tuned perfectly, but a corridor-wide green wave would reduce total delay far more; changing one signal alone worsens things.
Plateau	A flat region where neighbours have equal cost.	Small timing changes give no change in waiting time during uniform light traffic, so the search has no direction.
Ridge	A narrow path of improvement not reachable by single-step moves.	Improvement needs several linked intersections to change together (coordinated offsets); changing one at a time makes things worse.

iii. How Simulated Annealing (SA) and Genetic Algorithms (GA) overcome them

Simulated Annealing: picks a random neighbour; if it is better it is accepted, and if worse it is accepted with probability $e^{-dE/T}$, where T (temperature) decreases gradually (cooling schedule). Early on, uphill (worse) moves let the system escape local optima and cross plateaus or ridges; later it behaves like hill climbing and settles. It needs little memory and is simple to

parallelize by region.

Genetic Algorithm: maintains a population of timing plans (chromosome = timing of all intersections). Fitness = inverse of cost. Selection chooses good plans, crossover combines good parts of different plans (e.g., good corridors), and mutation randomly alters timings to add diversity. Because it explores many regions simultaneously, it avoids premature convergence and handles ridges through crossover of coordinated segments.

iv. Recommendation

Recommended: a hybrid approach, with a Genetic Algorithm (or a distributed GA by city zone) for global planning and Simulated Annealing / hill climbing for fast local fine-tuning.

- **Scalability:** GA evaluates many candidates in parallel and can divide the city into zones; SA scales with low memory; plain hill climbing does not escape bad optima.
- **Adaptability:** as traffic changes, the previous best population is reused as a warm start, so the system adapts without restarting; SA's mutation-like randomness keeps exploring.
- **Practicality:** run offline GA on historical patterns to build good baseline plans, then apply SA or hill climbing online for real-time tweaks using live sensor data.

Question 3: Autonomous Mars Rover (Online Search)

i. Why online search instead of offline search

Offline search computes a full plan before acting and assumes a known, static map and deterministic actions. The Mars rover does not have a complete map, hazards (craters, rocks) are discovered only when sensed, communication with Earth has a long delay, and the environment can change. Therefore the rover must **interleave planning and acting**: take a step, sense, update knowledge, and decide again. This is online search, where the agent knows only the local neighbourhood, discovers states by visiting them, and cannot compute the whole solution in advance.

ii. Challenges of partial observability and dynamic environments

- **Limited sensing:** the rover sees only a small area; hidden craters or soft sand may be undetected, so the state is uncertain.
- **Sensor noise and error:** readings may be inaccurate, requiring probabilistic estimates (e.g., localization).
- **Irreversible actions:** falling into a crater or getting stuck is irreversible and unrecoverable (unsafe exploration).
- **Dead ends and loops:** without a map the rover may revisit the same cells; it needs memory of visited states.
- **Dynamic factors:** dust storms, shifting sand and changing lighting alter terrain traversability and visibility.
- **Resource limits:** battery, computing power and communication restrict how much can be sensed and computed.
- **Exploration vs exploitation:** balance exploring new areas against reaching sample targets safely.

iii. How the rover builds and updates its internal model

- Starts with an empty or partial map (e.g., an occupancy grid with all cells unknown).
- Each sensing step (cameras, LiDAR, proximity, inclination) marks nearby cells as free, obstacle or hazardous, with a confidence value.
- It stores **RESULT[s, a] = s'** (the outcome of each action in each state) and a table of visited states/costs, as in online DFS or LRTA*.
- The model is updated with new observations (e.g., Bayesian or probabilistic updates), merging them with the existing map; localization (SLAM) keeps the rover's position consistent.
- Heuristic values are revised (as in LRTA*, where h(s) is updated based on experience), so the rover learns better estimates and avoids loops.
- Planning is repeated on the updated model: choose the next safe, useful move toward the sample site, then act and sense again.

iv. Comparison of online search with uninformed and informed search

Feature	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online search (online DFS, LRTA*, D* Lite)
Knowledge of environment	Full map assumed	Full map plus heuristic assumed	Unknown; learns while exploring
Planning vs acting	Plans completely, then acts	Plans completely, then acts	Interleaves planning and acting
Use of heuristic	None	Estimates cost to goal	Learns or updates heuristic from experience
Feature	**Uninformed (BFS, DFS, UCS)**	**Informed (Greedy, A*)**	**Online search (online DFS, LRTA*, D* Lite)**
Handles surprises	Poorly; must restart	Poorly; replan from scratch	Designed for it; adapts continuously
Safety	Not considered	Only if costs include hazards	Can be risk-aware and cautious
Suitability for Mars	Low	Moderate (if map known)	High

v. Most suitable approach

Recommended: an online informed search such as LRTA* or D* Lite (incremental A*), combined with a probabilistic map of the terrain and safety constraints.

- **Uncertainty:** a probabilistic internal map (occupancy grid with confidence) represents what is unknown.
- **Adaptability:** D* Lite repairs the existing plan efficiently when new obstacles are sensed instead of recomputing everything; LRTA* learns by updating heuristic values.
- **Safety:** assign high cost to unknown or risky cells, prefer reversible moves, keep safe margins from craters, and halt or retreat when uncertainty is too high.
- This gives the efficiency of informed search while still working with an incomplete map.

Question 4: University Exam Timetable Generation (Constraint Satisfaction Problem)

CSP formulation: variables, domains and constraints (i)

A CSP is defined as (X, D, C): a set of variables, their domains, and constraints over them.

Component	Definition for the exam timetable	Explanation
Variables (X)	One variable per exam: E1, E2, ..., En (one for each course).	Each exam must be assigned a time slot and a hall.
Domains (D)	Possible (time slot, hall) pairs, e.g., Day 1-10 x Slots 1-3 x Halls H1...Hm.	Domains can be reduced by faculty availability and student accommodations (e.g., ground-floor hall, extra time).
Constraint 1: No clash	If courses Ei and Ej share any student, then slot(Ei) is not equal to slot(Ej).	Binary constraint; prevents a student having two exams at the same time.
Constraint 2: Hall capacity	Number of exams in a slot is at most the number of halls, and students in a hall are at most the hall capacity.	Global resource constraint.
Constraint 3: Precedence	If Ei must come before Ej, then slot(Ei) is earlier than slot(Ej).	Binary ordering constraint (e.g., prerequisite or practical before theory).
Constraint 4: Faculty	An invigilator or examiner cannot be assigned to two exams in the same slot or outside availability.	Unary and binary constraints.
Constraint 5: Accommodations	Special students need specific halls, extra time or rest gaps.	Unary constraints that prune domains; rest gaps are hard or soft constraints.

Hard constraints must always be satisfied; soft preferences (e.g., avoid two exams on consecutive days, spread exams evenly) can be optimized afterwards. A solution is a complete and consistent assignment of all exams.

ii. How backtracking search works

- Select an unassigned variable (exam) and try a value (slot, hall) from its domain.
- Check consistency with the constraints against already assigned exams.
- If consistent, recurse to the next variable; if all values fail, **backtrack** to the previous variable and try its next value.
- Continue until all exams are assigned (solution) or the search space is exhausted (no solution).
- Example: assign Maths to Day 1 Slot 1 Hall A; the next exam Physics shares students with Maths, so Slot 1 is disallowed; try Slot 2, and so on. If Chemistry later has no valid slot, undo the most recent choices and try alternatives.

Plain backtracking is complete but can have exponential time because it discovers failures late and may repeat the same failures (thrashing).

iii. Constraint propagation and forward checking

Forward checking: whenever a variable is assigned, delete inconsistent values from the domains of its unassigned neighbours. If any domain becomes empty, backtrack immediately. For instance, after fixing Maths on Day 1 Slot 1, that slot is removed from the domains of all exams sharing students with Maths; if some exam is left with no slot, the assignment fails at once rather than later.

- Detects dead ends early and prunes large parts of the search tree.
- **Arc consistency (AC-3)** goes further: it propagates reductions between pairs of variables, catching failures that forward checking misses.

- Combine with heuristics: **MRV** (choose the exam with the fewest legal slots), **degree heuristic** (the exam with the most conflicts) and **LCV** (try the value that rules out the fewest options for others).

iv. Real-world challenges and best strategy for scalability

- **Scale:** thousands of courses and students create huge conflict graphs and domains.
- **Conflicting or over-constrained requirements:** a perfect solution may not exist, so soft constraints and priorities are needed.
- **Changes:** late faculty unavailability or new registrations require re-solving quickly.
- **Fairness and quality:** balanced exam spacing for students and invigilator workload.
- **Special cases:** accommodations, multiple campuses, hall capacity variations.

Best strategy: model as a CSP and solve with **backtracking + MRV/degree heuristics + forward checking / arc consistency**, giving a feasible timetable fast. For large instances, add **local search**
(min-conflicts) or metaheuristics (simulated annealing, genetic algorithms) to repair or improve solutions, decompose the problem (e.g., by department or by day), and use incremental re-solving for changes. This hybrid keeps hard constraints satisfied while scaling and optimizing soft preferences.

Question 5: Real-Time Strategy Game AI (Minimax and Alpha-Beta Pruning)

i. How Minimax models the problem

Minimax models the game as a two-player, zero-sum, adversarial search tree. Nodes are game states, edges are legal moves, and levels alternate between MAX (the AI, trying to maximize its utility) and MIN (the human, assumed to play the best opposing move). Leaf (terminal or depth-limit) states receive utility values; values propagate upward: MAX nodes take the maximum of their children, MIN nodes take the minimum. At the root, the AI picks the move leading to the best guaranteed outcome against an optimal opponent. Minimax assumes perfect information and a rational opponent; in practice, the real-time game is approximated by discretizing time and using a depth limit with an evaluation function.

ii. Computational challenges of large game trees

- **Exponential growth:** with branching factor b and depth d, Minimax visits $O(b^d)$ nodes. Strategy games have dozens or hundreds of possible actions (move units, build, attack), so the tree is astronomically large.
- **Strict time limits:** a decision is required within milliseconds to seconds, so full-depth search is impossible.
- **Memory:** storing states and values is expensive; complex state evaluation is costly at each node.
- **Continuous and simultaneous actions:** real-time games lack clean turns; hidden information (fog of war) and randomness break the perfect-information assumption.
- Result: the AI must cut off search early and use heuristics.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta keeps two bounds during the search: **alpha** = the best value MAX can already guarantee along the path, and **beta** = the best value MIN can already guarantee. If at any node beta is less than or equal to alpha, the remaining children of that node cannot influence the final decision and are **pruned**.

Reasoning: suppose MAX has found a move worth 5. While exploring another move, MIN finds a reply that gives only 3. MIN would pick that reply (or even better for MIN), so this move is worth at most 3, which is already worse than 5; MAX will never choose it, so MIN's other replies need not be examined.

- **The result is identical to Minimax** (same optimal move), but with fewer nodes evaluated.
- **Best case** (good move ordering): complexity is $O(b^{d/2})$, meaning the search can go about **twice as deep** in the same time; **worst case** (poor ordering): $O(b^d)$, no gain.
- **Move ordering** is essential: try captures, strong attacks and previous best moves first; use **iterative deepening** and killer-move or history heuristics, plus transposition tables to avoid re-evaluating repeated states.

iv. Evaluation function design

When the search is cut off at a depth limit, a heuristic evaluation function estimates how good a state is for the AI: **Eval(s) = w1 f1(s) + w2 f2(s) + ... + wn fn(s)** (a weighted linear combination of features, with weights tuned by testing or learned from data).

Factor (feature)	Description	Justification
Material / unit strength	Difference between own and enemy army value (unit count, health, type).	Direct indicator of combat power and winning chances.
Resources and economy	Stockpile, income rate, number of workers or bases.	Economic strength converts to future army and tech.
Factor (feature)	**Description**	**Justification**
Map and territory control	Controlled bases, chokepoints, vision area.	Position gives tactical advantage and information.
Technology and upgrades	Unlocked tech tree and upgrades.	Long-term strength and unit effectiveness.
Threat and safety	Enemy units near own base, vulnerability of key buildings.	Avoids defeats; protecting the base or commander is crucial.
Mobility / tempo	Unit positioning, ability to attack or retreat.	Better flexibility yields initiative.

The function must be fast to compute (called at every leaf), consistent with the true utility at terminal states (win = large positive, loss = large negative), and the weights are adjusted for game phase (early
economy-heavy, late army-heavy).

v. Strategy balancing optimality and real-time performance

Recommended: Alpha-Beta search with iterative deepening, a time budget, good move ordering and a heuristic evaluation function.

- **Iterative deepening with time control:** search depth 1, 2, 3, ... until time runs out, and play the best move from the last completed depth (an anytime algorithm).
- **Depth-limited search + evaluation function:** trades perfect optimality for speed.
- **Move ordering, transposition tables, and pruning of unpromising moves** (forward pruning or beam search) cut the effective branching factor.
- **Abstraction:** group units into squads or macro-actions, reducing the action space.
- **Complement with other techniques** when the tree is too large: Monte Carlo Tree Search (MCTS) for stochastic, hidden-information situations, and precomputed opening books or learned policies.
- **Adaptive difficulty:** adjust depth or time budget to fit hardware and player level.

This gives near-optimal decisions within strict time limits while keeping the AI competitive against human players.